

John Zachary Fitch

Agent tooling | systems performance | privacy-first infrastructure

I build production-grade tooling for agents and the substrate they depend on: deterministic retrieval, verifiable edits, structured tool APIs, and execution environments you can reason about. I work across Rust, Python, and web platforms with an evidence-first style: measure, reproduce, fix, and ship.

Recent Highlight (Jan 2026)

OPENAI CODEX

Ghost in the Codex Machine

Investigated an "invisible" release-only regression where a pre-main constructor stripped `LD_*` / `DYLD_*` environment variables, breaking CUDA/MKL library discovery inside tool subprocesses. My investigation led to the fix in rust-v0.80.0, and the release notes thank me by name.

PROOF

 [Issue #8945](#) - [Fix PR #8951](#)

 [Fixed in rust-v0.80.0, with release-notes credit](#)

Representative impact

- MKL/BLAS (repro harness): ~2.71s -> ~0.239s (11.3x)
- CUDA workflows: restored expected library discovery to avoid CPU fallback in affected setups

Release notes excerpt:

"Special thanks to @johnzfitch for the detailed investigation and write-up in #8945."

Research

Independent research on the internal geometry of transformers, and on model architectures built from the exceptional Jordan algebra (the 27-dimensional Albert algebra).

- Built the Albert Engine, a tested numerical runtime for the Albert algebra, used to prototype geometry-aware sequence models, memory mechanisms, and optimizers.
- Run causal interventions on pretrained open models to test which geometric structure they actually depend on.

- Register predictions before experiments, label each claim as proved, measured, or conjectured, and report negative results alongside positive ones.

[Research page](#)

How I Work

- Construct from first principles rather than refine existing forms.
- Recover overlooked work — revisit ideas and approaches that didn't take the first time, with current tools.
- Probe working systems for hidden assumptions; rebuild the parts that don't hold up.
- Operate well in complex, debated, high-ambiguity spaces.
- Find a useful subspace inside a team early; manage well under crisis and time pressure.

Core Skills

Rust

Performance-critical systems, CLI tools, data structures, correctness-oriented engineering

Python

Tooling, analysis pipelines, automation, reproducible experiments

Web

WebGPU/WASM applications, client-side ML inference, offline-first UX

Systems

Linux, NixOS, DNS, TLS automation, containerized services, security hardening

Agent Integration

MCP servers, skill/plugin packaging, tool-driven workflows

Selected Projects (Public)

- **[llmx](#)** (Rust core, JS/WASM web; live: [llm.cat](#)) - local-only codebase indexer built on Burn-ml and mdbr-leaf-ir; hybrid search (BM25 + neural embeddings) fused via RRF; deterministic chunking with content hashing
- **[claude-warden](#)** (Shell/OTEL, ★60+) - security hooks and budget system for Claude Code, with a built-in web viewer for extensive OTEL traces
- **[claude-cowork-linux](#)** (Linux, ★400+) - native Linux port of Claude Desktop, security-first: treat the host OS as the VM (no Mac VM layer), wrap in bubblewrap, and handle the ASAR from outside the sandbox rather

than hooking in; strips dispatch/channels, preserves Cowork/Code/Chat

- **dota** (Rust) - post-quantum secrets manager: hybrid ML-KEM-768 + X25519, Argon2id KDF, SQLCipher at rest, YubiKey/SoloKey HMAC-SHA1 hardware auth
- **pyghidra-lite** (Python/MCP, ★35+) - token-efficient MCP server for Ghidra (official MCP registry: io.github.johnzfitch/pyghidra-lite , v0.1.1)
- **codex-xtreme** (Rust) - optimized, patched Codex builds (includes [codex-patcher](#))
- **burn-plugin** - Claude Code plugin + skills for the Burn deep learning framework
- **Observatory** (WebGPU) - client-side AI image detection (live)
- **SpecHO v2** (Python) - 161D linguistic fingerprinting for AI text detection
- **definitelynot.ai** (PHP/JS) - Unicode-security-aware sanitizer + API
- **Iconics** (Python) - semantic icon library for professional docs (8k+ icons)

Operating Background

Owner-operator of a regulated chemical-manufacturing business for nearly nine years; regulated as a tobacco product by FDA, with hazardous-substance handling. Started in 2014 selling hand-mixed e-liquid on Reddit's e-cigarette classifieds; built the brand ("The Best Damn Liquids," federally trademarked) on made-to-order mixing, a panel of about 50 outside flavor testers, and a \$11.99-per-30ml price ceiling, serving roughly 40,000 customers by 2019 and more than 200,000 orders with a team of up to six. Filed a Premarket Tobacco Product Application before FDA's September 2020 deadline and worked through years of non-committal guidance, including FOIA requests; FDA refused to file the application. Mission: help adults worldwide quit smoking cigarettes. Declined paths that diverged from that mission even when commercially favorable; closed in July 2023, the company's strongest revenue year.

What this established:

- Tolerance for prolonged regulatory ambiguity without losing direction.
- Operating discipline at production scale: manufacturing, hiring, fulfillment, crisis response.
- Nearly nine years under public scrutiny — grounded in the work, confident defending it on the record, practiced at navigating it responsibly.

Infrastructure (Self-Hosted)

I operate production infrastructure on bare metal with:

- Declarative NixOS configuration (reproducible, atomic upgrades, rollbacks)
- Authoritative DNS and automated wildcard certificates (DNS-01 / RFC2136)
- Post-quantum security layers (hybrid SSH KEX, WireGuard + Rosenpass)

Education

UC Berkeley - B.A. in Mathematics, May 2022.

Santa Rosa Junior College - A.S. in Mathematics and an associate degree in Economics, August 2020.

Origin

The research question started on two pages of notes from a linear algebra course at SRJC in 2019: if two brains carry out the same functions in different places, is there a map that sends each onto shared functional pieces where the difference becomes readable? This research asks the same question of transformers.

What I'm Looking For

Roles building agent runtimes and developer tools, retrieval systems, and security/privacy foundations. I work best on teams that value measurable results, clear ownership, and high engineering standards.